

Sistema IoT de Seguridad Perimetral con IA Basica

Documento final editable para entrega formal

Alcance resuelto

El presente documento desarrolla integralmente el Reto 2 del proyecto IoT. Incluye tres patrones arquitectonicos aplicados, diagrama C4 nivel 3 de componentes, especificacion de un prototipo funcional de punta a punta, tabla de trade-offs y guion detallado para video demo de 5 minutos.

Elemento	Contenido
Curso	Arquitectura de Software + IoT
Unidad	Unidad 2 - Estilos, Patrones y Diseno Tactico
Entregable	Reto 2 - Patrones + Implementacion
Proyecto	Sistema IoT de seguridad perimetral con IA basica
Foco	Patrones, C4 nivel 3, prototipo funcional, trade-offs y video demo

Este reporte esta planteado como una entrega autocontenida: toma el proyecto IoT ya definido y lo evoluciona a nivel tactico sin depender de anexos adicionales para comprender la arquitectura propuesta.

1. Resumen ejecutivo

La evolucion del sistema se centra en convertir una arquitectura IoT conceptual en un prototipo funcional y defendible desde el punto de vista arquitectonico. Para ello se consolidan tres patrones con impacto directo en la operacion del sistema: Pub-Sub como espina dorsal de mensajeria, Gateway para concentrar integracion y reglas en Node-RED, y Repository para encapsular la persistencia de eventos, alertas y evidencia.

El resultado esperado es un flujo operativo repetible y de bajo costo: un sensor real o simulado publica un evento de movimiento; el broker MQTT lo distribuye; Node-RED lo valida, clasifica y persiste; el dashboard presenta el incidente y el operador consulta su trazabilidad historica. La solucion sigue siendo compatible con ESP32-CAM + PIR, pero se documenta de forma que tambien pueda demostrarse mediante simulacion local para garantizar repetibilidad en la entrega.

Criterio rector del documento

La entrega queda alineada con los cinco puntos evaluados en el Reto 2: patrones aplicados, C4 nivel 3, implementacion funcional, trade-offs y video demo.

1.1 Trazabilidad de la entrega frente al Reto 2

Requisito evaluado	Desarrollo dentro del documento	Evidencia entregable
3 patrones arquitectonicos	Seccion 2	Justificacion, rol en el sistema y relacion con atributos de calidad
C4 nivel 3 - Componentes	Seccion 3	Diagrama de componentes y tablas por contenedor
Implementacion funcional	Seccion 4	Topologia, topicos MQTT, flujo Node-RED, persistencia y validacion
Tabla de trade-offs	Seccion 5	Tres compensaciones reales con impacto en calidad
Video demo 5 minutos	Seccion 6	Guion minuto a minuto, checklist y estructura GitHub

1.2 Supuestos de implementacion

- El prototipo se documenta con sensor simulado compatible con el mismo contrato MQTT que usaria un ESP32-CAM + PIR; por tanto, el publicador puede reemplazarse sin alterar el resto de la arquitectura.
- Se asume un despliegue local o de laboratorio con broker MQTT, Node-RED, una base SQLite y dashboard en la misma red.
- La notificacion externa (correo o Telegram) se modela como integracion opcional; el flujo minimo exigido queda completamente cubierto con sensores -> broker MQTT -> Node-RED -> dashboard.
- La persistencia prioriza trazabilidad y simplicidad: append de eventos, registro de alertas validadas y metadatos de evidencia, con posibilidad de migrar a PostgreSQL sin reescribir la logica de negocio.

2. Tres patrones arquitectonicos aplicados

La seleccion de patrones responde a tres necesidades dominantes del sistema: desacoplar la comunicacion entre dispositivos y consumidores, centralizar la integracion con una logica de negocio modificable, y

persistir la informacion critica sin contaminar los flujos de procesamiento con detalles de acceso a datos. A partir de esas necesidades se adoptan Pub-Sub, Gateway y Repository.

Patron	Problema que resuelve	Aplicacion concreta en el sistema	Atributos de calidad reforzados
Pub-Sub	Eventos asincronos y multiples consumidores sin acoplamiento fuerte	Sensores publican por MQTT; Node-RED, dashboard y modulos futuros se suscriben por topico	Escalabilidad, latencia, modificabilidad
Gateway	Heterogeneidad de dispositivos y necesidad de un punto unico de integracion	Node-RED recibe, normaliza, clasifica, expone dashboard y encapsula integraciones	Modificabilidad, usabilidad operativa, seguridad
Repository	Necesidad de auditoria y persistencia desacoplada de la logica	Subflujos o servicios de repositorio para eventos, alertas, evidencia y estado de zona	Trazabilidad, mantenibilidad, testabilidad

2.1 Patron Pub-Sub

El sistema de seguridad perimetral produce eventos de manera asincrona: un sensor detecta movimiento en un instante impredecible y, a partir de ahi, varios consumidores pueden necesitar ese mismo evento. Un consumidor lo usa para validar la alerta, otro para persistirla, otro para presentarla al operador y, en una evolucion futura, otro para analitica o mantenimiento predictivo. Si los dispositivos tuvieran que conocer a todos los consumidores, el firmware se volveria fragil y costoso de mantener.

La aplicacion concreta del patron se realiza sobre MQTT. Cada nodo publica en un namespace de topicos por zona y tipo de dato; por ejemplo, `iot/perimetro/zona1/movimiento/raw`. Node-RED se suscribe a la capa raw para procesar eventos entrantes, mientras que el dashboard y otros consumidores internos atienden topicos ya clasificados, como `iot/perimetro/alertas/validada` o `iot/perimetro/estado/zona1`. De esta forma, el sensor solo conoce el broker y el contrato del mensaje, no a los consumidores especificos.

- Permite agregar nuevas zonas, sensores o consumidores sin modificar los publicadores existentes.
- Disminuye el acoplamiento temporal: el productor publica y continua, aun si algun consumidor se reconecta despues.
- Se adapta bien a infraestructura economica porque MQTT consume poco ancho de banda y mantiene la operacion simple.
- Facilita la evolucion del prototipo hacia escenarios con analitica, almacenamiento distribuido o microservicios especializados.

2.2 Patron Gateway

En esta iteracion, Node-RED deja de ser solo un motor de reglas y asume explicitamente el rol de Gateway de aplicacion. Ese Gateway recibe trafico MQTT desde los nodos edge, valida contratos, normaliza payloads, encapsula la logica de IA basica, actualiza el dashboard, despacha notificaciones y sirve como punto de entrada coherente para futuras integraciones HTTP o WebSocket. El beneficio principal es que la inteligencia operativa queda concentrada fuera del dispositivo, con cambios rapidos y de bajo riesgo.

El patron es especialmente adecuado para IoT porque evita dispersar reglas de negocio en hardware limitado. El nodo ESP32 o el simulador solo detectan, empaquetan y publican; el Gateway se ocupa de decidir si un evento se descarta, se marca como sospechoso o se convierte en una alerta formal. Tambien protege al resto del sistema de variaciones de dispositivos, ya que cualquier cambio de payload o de topicos se absorbe en la capa de normalizacion del Gateway.

- Centraliza autenticacion de mensajes, validacion de esquema y politicas operativas.
- Reduce el tiempo de cambio, porque nuevas reglas se agregan en Node-RED y no en firmware distribuido.

- Permite exponer el dashboard y la consulta historica sin conectar la interfaz directamente al broker ni a la base de datos.
- Sirve como punto de integracion para notificaciones externas y futura automatizacion de comandos hacia los dispositivos.

2.3 Patron Repository

El sistema necesita trazabilidad completa: saber que evento entro, como fue clasificado, que evidencia se asocio y cual es el estado actual de la alerta. Si cada flujo de Node-RED realizara sentencias SQL directas y dispersas, el mantenimiento se volveria inconsistente y fragil. Para evitarlo se adopta el patron Repository mediante subflujos o servicios reutilizables que encapsulan el acceso a datos.

En el prototipo se definen cuatro repositorios logicos: EventRepository, AlertRepository, EvidenceRepository y ZoneStateRepository. La logica de negocio invoca operaciones de alto nivel, como guardarEventoRaw, crearAlertaValidada o actualizarEstadoZona, sin conocer detalles de SQLite o PostgreSQL. Esto mejora la mantenibilidad, simplifica pruebas y deja preparada una evolucion futura hacia una base de datos mas robusta.

- Garantiza consistencia en nombres de campos, timestamps y estados de alerta.
- Aisla cambios de tecnologia de persistencia respecto de los flujos de negocio.
- Facilita auditoria, reportes y consultas historicas del dashboard.
- Permite incorporar validaciones transversales, por ejemplo idempotencia o versionado de registros, en un solo punto.

2.4 Articulacion de los patrones en una sola solucion

Decision	Papel dentro del sistema	Efecto esperado en la operacion
Pub-Sub	Define el tejido de comunicacion entre sensores, procesamiento y consumidores	Eventos desacoplados y posibilidad de crecer por zona o por tipo de consumidor
Gateway	Concentra la coordinacion tactica, la validacion de reglas y la exposicion al dashboard	Cambios rapidos en logica sin tocar hardware ni consumidores externos
Repository	Estabiliza la persistencia y la consulta historica	Auditoria consistente y menor dependencia de una base de datos puntual

3. Diagrama C4 nivel 3 - Componentes

El nivel 3 del modelo C4 desglosa los contenedores principales del sistema en componentes internos con responsabilidades concretas. La descomposicion propuesta conserva la coherencia del diseno general y muestra donde viven los tres patrones seleccionados: MQTT materializa Pub-Sub, Node-RED opera como Gateway y la base de datos se accede a traves de Repository.

3.1 Criterios de descomposicion

- Mantener liviano el nodo edge: solo captura, construye el evento y publica.
- Evitar que el broker contenga logica de negocio; su responsabilidad es mensajeria y control de acceso.
- Concentrar en Node-RED la logica operativa de validacion, clasificacion y actualizacion de vistas.
- Separar la interfaz de consulta historica de los mecanismos de persistencia para preservar bajo acoplamiento.
- Representar los componentes de manera suficientemente detallada para guiar una implementacion, sin sobrediseñar el prototipo.

C4 nivel 3 - Componentes del sistema

Proyecto IoT de seguridad perimetral con IA basica

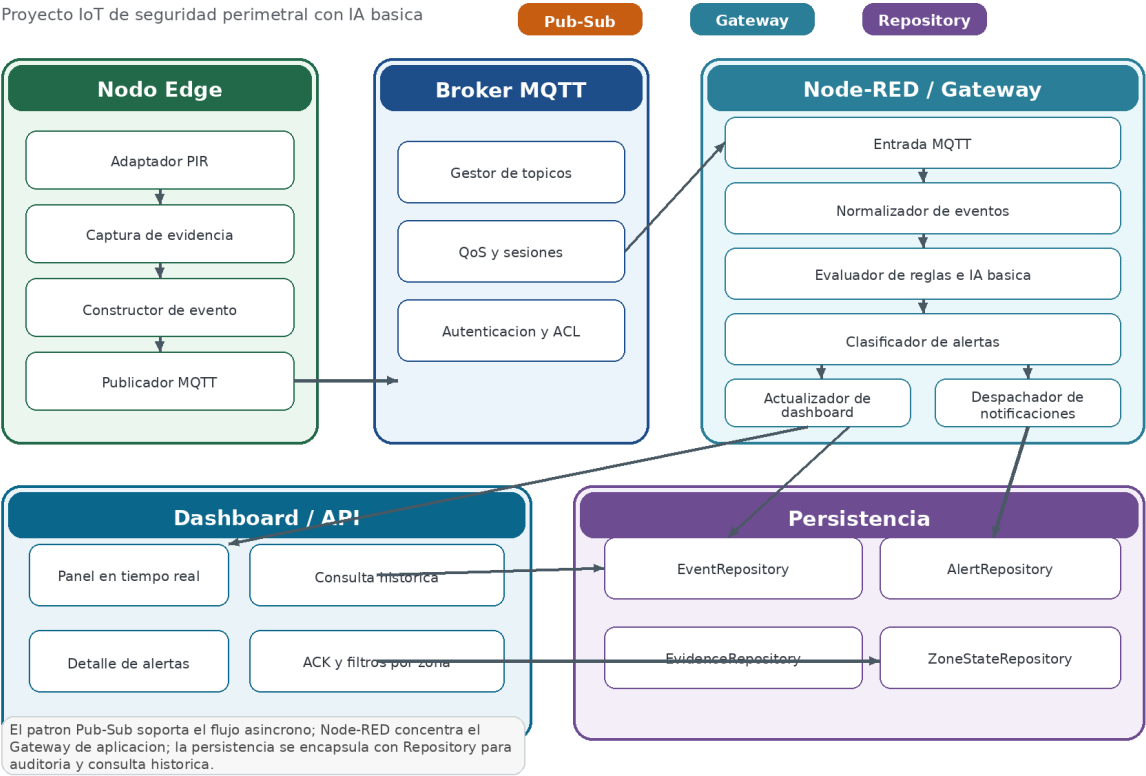


Figura 1. C4 nivel 3 de componentes del sistema IoT de seguridad perimetral

3.2 Desglose de componentes por contenedor

Contenedor A - Nodo Edge

Componente	Responsabilidad	Entradas / salidas principales
Adaptador PIR	Lee el estado del sensor de movimiento y genera un trigger simple	Entrada fisica del PIR -> bandera motion=true
Captura de evidencia	Toma una imagen o referencia de captura cuando ocurre un trigger	Trigger de movimiento -> imageRef o metadata de evidencia
Constructor de evento	Arma el payload JSON con deviceId, zona, timestamp y metadatos	Datos del sensor + evidencia -> evento normalizado de borde
Publicador MQTT	Envia el evento al broker usando el topico raw correspondiente	Evento construido -> publicacion MQTT con QoS definido

Contenedor B - Broker MQTT

Componente	Responsabilidad	Entradas / salidas principales
Gestor de topicos	Administra namespace, rutas y suscripciones por zona y tipo de mensaje	Publicaciones y suscripciones MQTT
QoS y sesiones	Controla entrega, reconexion y comportamiento de clientes	Mensajes MQTT -> entrega a consumidores
Autenticacion y ACL	Aplica credenciales y permisos sobre topicos autorizados	Conexion del cliente -> aceptacion o rechazo

Contenedor C - Node-RED / Gateway

Componente	Responsabilidad	Entradas / salidas principales
Entrada MQTT	Se suscribe a la capa raw y recibe eventos desde el broker	Topicos raw -> mensaje interno Node-RED
Normalizador de eventos	Valida esquema, homogeneiza nombres y completa campos faltantes	Mensaje raw -> payload canonico
Evaluador de reglas e IA basica	Calcula score de confianza y reduce falsos positivos	Payload canonico -> evento evaluado
Clasificador de alertas	Decide si el evento se ignora, se registra solo o se convierte en alerta	Evento evaluado -> estado operativo
Actualizador de dashboard	Publica estado resumido y ultimo incidente al panel	Alerta o evento -> vista en tiempo real
Despachador de notificaciones	Emite aviso a correo o Telegram cuando la politica lo exige	Alerta validada -> mensaje externo
Adaptador de repositorio	Invoca operaciones de persistencia sin exponer SQL en los flujos	Evento o alerta -> escritura o consulta

Contenedor D - Dashboard / API

Componente	Responsabilidad	Entradas / salidas principales
Panel en tiempo real	Muestra ultima alerta, estado por zona y actividad reciente	Estado publicado por Gateway -> visualizacion
Detalle de alertas	Presenta metadata de cada incidente y evidencia asociada	Alertas validadas -> detalle operativo
Consulta historica	Permite revisar eventos y alertas almacenadas	Filtros del usuario -> consultas al repositorio
ACK y filtros por zona	Marca alertas atendidas y mejora el trabajo del operador	Accion del usuario -> actualizacion de estado

Contenedor E - Persistencia

Componente	Responsabilidad	Entradas / salidas principales
EventRepository	Conserva el rastro completo de eventos entrantes, incluso los no escalados	Evento raw o normalizado -> registro auditable
AlertRepository	Guarda las alertas validadas, severidad y estado operativo	Alerta validada -> historial y seguimiento
EvidenceRepository	Mantiene metadatos de capturas o archivos asociados	imageRef -> evidencia vinculada
ZoneStateRepository	Sostiene el ultimo estado por zona para la vista en vivo	Cambios de evento -> estado actual por zona

3.3 Relaciones criticas entre componentes

Relacion	Justificacion arquitectonica
----------	------------------------------

Constructor de evento -> Publicador MQTT	Separa captura fisica de transporte para no mezclar logica de dispositivo y comunicacion.
Broker MQTT -> Entrada MQTT	Materializa el desacoplamiento Pub-Sub: el broker media sin conocer reglas de negocio.
Normalizador -> Evaluador -> Clasificador	Establece un pipeline claro donde cada paso cumple una sola responsabilidad.
Clasificador -> Adaptador de repositorio	La persistencia se activa por decisiones de negocio y se encapsula tras Repository.
Actualizador de dashboard / Consulta historica	El operador dispone simultaneamente de estado en vivo y trazabilidad persistente.

4. Implementacion funcional del prototipo

La implementacion funcional se diseña para cumplir el flujo exigido por el Reto 2: sensores reales o simulados -> broker MQTT -> Node-RED -> dashboard. El enfoque elegido privilegia repetibilidad, bajo costo y facilidad de demostracion. Por eso, el prototipo puede ejecutarse con un publicador simulado durante la demo y conservar compatibilidad con un nodo ESP32-CAM + PIR cuando se disponga del hardware.

Flujo funcional de punta a punta

Sensores reales o simulados -> MQTT -> Node-RED -> repositorio -> dashboard

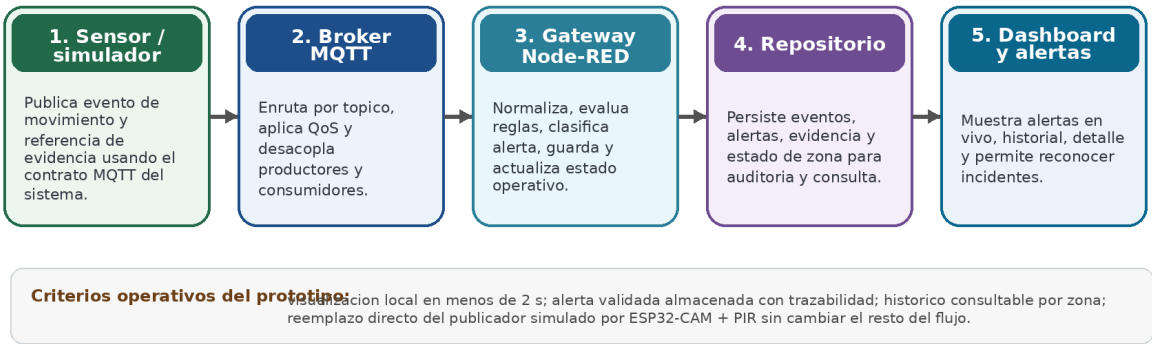


Figura 2. Flujo funcional propuesto para la demostracion punta a punta

4.1 Entorno y stack de implementacion

Capa	Tecnologia sugerida	Rol dentro del prototipo	Razon de seleccion
Sensores	ESP32-CAM + PIR o simulador Python	Genera eventos compatibles con el contrato MQTT	Permite demo con o sin hardware real
Mensajeria	Mosquitto	Broker MQTT para publicacion y suscripcion	Ligero, estable y ampliamente usado en IoT
Gateway	Node-RED	Recibe, valida, orquesta y actualiza vistas	Acelera integracion y cambio de reglas
Persistencia	SQLite en prototipo	Conserva eventos, alertas, evidencia y estado	Simple, embebido y suficiente para laboratorio
Visualizacion	Node-RED Dashboard	Presenta estado en vivo e historial	Rapidez de entrega y bajo costo de configuracion
Notificacion opcional	Correo o Telegram	Complementa la alerta visual del dashboard	Agrega valor operativo sin alterar el flujo minimo

4.2 Contratos de mensajeria MQTT

Topico	Publicador	Consumidor	Proposito	QoS
iot/perimetro/+/movimiento/raw	Nodo edge o simulador	Gateway Node-RED	Eventos de movimiento sin procesar	1
iot/perimetro/+/evidencia/raw	Nodo edge o simulador	Gateway Node-RED	Referencia o metadata de evidencia	0
iot/perimetro/alertas/validada	Gateway Node-RED	Dashboard y notificaciones	Alertas oficiales del sistema	1

iot/perimetro/estado/+	Gateway Node-RED	Dashboard	Ultimo estado resumido por zona	0
iot/perimetro/comandos/+	Dashboard o administracion	Nodo edge	Comandos futuros de configuracion o armado	1

Payload de evento de movimiento

```
{
  "deviceId": "esp32cam-z1",
  "zone": "zona1",
  "eventType": "motion",
  "motion": true,
  "confidence": 0.84,
  "imageRef": "capture_20260414_203000.jpg",
  "timestamp": "2026-04-14T20:30:00-05:00"
}
```

Payload de alerta validada

```
{
  "alertId": "ALT-20260414-0001",
  "deviceId": "esp32cam-z1",
  "zone": "zona1",
  "severity": "alta",
  "finalScore": 0.82,
  "status": "nueva",
  "evidenceRef": "capture_20260414_203000.jpg",
  "createdAt": "2026-04-14T20:30:02-05:00"
}
```

4.3 Logica de procesamiento en Node-RED

Paso	Nodo o subflujo sugerido	Responsabilidad
1	mqtt in	Escucha topicos raw de movimiento y evidencia
2	json	Convierte el payload entrante a objeto manipulable
3	function - normalize	Valida campos minimos y completa defaults
4	function - score	Calcula score de confianza con reglas de IA basica
5	switch	Separa eventos descartados, sospechosos o alertas validadas
6	subflow - repositories	Guarda evento, alerta, evidencia y estado de zona
7	ui_text / ui_table / ui_template	Actualiza el dashboard en tiempo real
8	http request o nodo de correo	Dispara notificacion opcional al canal externo

4.4 IA basica y reglas de validacion

La IA de esta iteracion no pretende ser un modelo entrenado de vision artificial, sino una heuristica explicable de bajo costo que reduzca falsos positivos. Se calcula un score final a partir de senales simples disponibles en el prototipo y se convierte en alerta solo aquello que supera un umbral operativo. La decision es consistente con el alcance academico y con la necesidad de mantener el sistema comprensible.

Factor	Regla	Impacto en el score
--------	-------	---------------------

Trigger PIR	Si motion=true	+0.55
Evidencia disponible	Si imageRef existe	+0.10
Consistencia temporal	Si hay segundo evento en la misma zona en <= 10 s	+0.20
Horario de vigilancia	Si ocurre dentro de la franja configurada de monitoreo	+0.15
Ventana de rebote	Si el evento cae en un cooldown muy corto	-0.20

Decision operativa

Se define un umbral de alerta validada igual o superior a 0.70. Con ello, eventos aislados o sin evidencia pueden registrarse para auditoria pero no saturan el dashboard con alertas innecesarias.

4.5 Persistencia y modelo de datos

Repositorio o tabla	Campos clave	Uso dentro del prototipo
events	id, device_id, zone, raw_payload, received_at	Bitacora de todo evento entrante; base para auditoria y debugging
alerts	alert_id, event_id, severity, final_score, status, created_at	Registro de alertas validadas y su seguimiento operativo
evidence	evidence_id, alert_id, file_name, file_path, created_at	Vinculo entre incidente y evidencia asociada
zone_state	zone, last_event_at, last_score, current_status	Vista sintetica para el dashboard en tiempo real

4.6 Dashboard operativo

- Tarjetas por zona con estado actual: normal, sospechoso o alerta.
- Panel de ultima alerta con hora, zona, score final y referencia de evidencia.
- Tabla de ultimos eventos y alertas consultables por zona.
- Vista de detalle para revisar evidencia, severidad y estado de atencion.
- Accion de reconocimiento (ACK) para que el operador marque una alerta como atendida.

4.7 Secuencia operativa de punta a punta

- El sensor real o el simulador detecta movimiento y construye el payload JSON del evento.
- El publicador envia el mensaje al broker en el topico raw correspondiente a la zona.
- Node-RED recibe el mensaje, lo convierte a JSON y valida que los campos minimos existan.
- El normalizador homogeneiza el contrato y agrega informacion necesaria para el procesamiento.
- El evaluador aplica la heuristica de IA basica y genera un score final.
- El clasificador decide si el evento se descarta, se conserva solo para auditoria o se eleva a alerta validada.
- El adaptador de repositorio guarda el evento y, si corresponde, crea la alerta y relaciona la evidencia.
- El actualizador de dashboard refresca el estado de la zona y muestra la alerta al operador.
- Opcionalmente se emite una notificacion externa si la politica de severidad lo exige.
- El operador consulta el historial o reconoce la alerta, manteniendo trazabilidad completa.

4.8 Casos de validacion y criterios de aceptacion

Caso	Estimulo	Comportamiento esperado	Criterio de aceptacion
------	----------	-------------------------	------------------------

TC-01	Publicacion de un evento valido de movimiento	El dashboard refleja la zona activa y el evento queda registrado	Visualizacion local en menos de 2 s
TC-02	Evento con score inferior al umbral	Se conserva en bitacora, pero no genera alerta visible de alta prioridad	No aparece en el canal de alertas validadas
TC-03	Evento con evidencia y score ≥ 0.70	Se crea alerta, se persiste, se actualiza la zona y se muestra el detalle	Alerta disponible con trazabilidad completa
TC-04	Consulta historica por zona	El operador puede recuperar el historial de eventos y alertas	Resultados consistentes desde el repositorio
TC-05	Payload malformado o incompleto	El Gateway rechaza o marca el mensaje sin comprometer el servicio	Error controlado y servicio operativo

4.9 Artefactos tecnicos listos para uso

Para facilitar la implementacion inmediata, el anexo del documento incluye comandos MQTT corregidos, un script minimo de publicacion simulada y la logica sugerida para el nodo Function de Node-RED. Con ello, el prototipo puede montarse rapidamente sin ambigüedades de contrato ni de procesamiento.

5. Tabla de trade-offs

Una arquitectura IoT util no se evalua solo por lo que gana, sino tambien por lo que sacrifica. Las siguientes compensaciones son reales dentro del diseno propuesto y muestran como se toman decisiones conscientes frente a calidad, costo y complejidad.

Decision arquitectonica	Lo que se gana	Lo que se sacrifica	Impacto en calidad y mitigacion
Pub-Sub con MQTT en lugar de integracion punto a punto	Desacoplamiento, facilidad de crecimiento y menor friccion para agregar consumidores	Mayor dificultad de trazado extremo a extremo y posibilidad de mensajes duplicados o desordenados	Mejora escalabilidad y latencia. Se mitiga con topicos consistentes, correlation IDs y bitacora append-only
Node-RED como Gateway central de negocio	Reglas modificables, integracion rapida y menor complejidad en los dispositivos	Se crea un hotspot operacional que debe cuidar rendimiento y disponibilidad	Mejora mantenibilidad y usabilidad. Se mitiga manteniendo flujos modulares y preparado el particionado por zonas
Repository sobre acceso SQL disperso	Persistencia uniforme, independencia de base de datos y mayor testabilidad	Se agrega una capa de abstraccion y un ligero costo inicial de modelado	Mejora trazabilidad y mantenibilidad. Se mitiga con subflujos simples y operaciones bien nombradas

5.1 Lectura de los trade-offs

Ninguna de las tres decisiones pretende optimizar simultaneamente todos los atributos de calidad. Se privilegia un equilibrio adecuado para una entrega academica con proyeccion realista: la mensajeria desacoplada ayuda a crecer; el Gateway reduce tiempo de cambio; y Repository protege la trazabilidad del

sistema. El costo es una arquitectura un poco mas sofisticada que la minima posible, pero significativamente mas defendible y evolucionable.

6. Video demo de 5 minutos

Aunque el video es un entregable independiente, este documento deja definido el guion tecnico completo para grabarlo y subirlo a GitHub. El objetivo de la demo es evidenciar que el sistema opera de punta a punta y que las decisiones arquitectonicas no son solo teoricas, sino visibles en el prototipo funcionando.

6.1 Objetivos de la demo

- Mostrar el flujo sensor o simulador -> broker MQTT -> Node-RED -> dashboard sin saltar etapas.
- Explicar brevemente donde se aplican Pub-Sub, Gateway y Repository.
- Evidenciar que la alerta queda almacenada y puede consultarse historicamente.
- Cerrar la demo mostrando la estructura del repositorio GitHub con los artefactos de soporte.

6.2 Guion minuto a minuto

Tiempo	Que mostrar	Mensaje clave a comunicar
0:00 - 0:30	Presentacion del proyecto y objetivo del sistema	Se trata de un sistema IoT de seguridad perimetral con flujo de alertas en tiempo real
0:30 - 1:10	Diagrama C4 nivel 3 y patrones aplicados	Pub-Sub desacopla, Node-RED actua como Gateway y Repository garantiza trazabilidad
1:10 - 2:10	Terminal o script publicando un evento MQTT	El sensor real o simulado usa el mismo contrato y topico
2:10 - 3:10	Node-RED recibiendo, normalizando y calculando score	La IA basica es explicable y filtra falsos positivos
3:10 - 4:00	Dashboard mostrando alerta y estado de la zona	La alerta es visible en vivo y esta asociada a su zona
4:00 - 4:35	Consulta historica o base de datos	La evidencia queda persistida con trazabilidad operativa
4:35 - 5:00	Cierre con trade-offs y carpeta GitHub	La arquitectura gana desacoplamiento, control y mantenibilidad con costos conocidos

6.3 Checklist antes de grabar

- Broker MQTT activo y escuchando en el host configurado.
- Flujo Node-RED cargado, desplegado y con salida visible en dashboard.
- Base de datos inicializada con tablas events, alerts, evidence y zone_state.
- Terminal lista con mosquitto_sub y mosquitto_pub o script Python publicador.
- Dashboard abierto en navegador y consulta historica preparada.
- Repositorio GitHub actualizado con README, diagrama, flujo y video final.

6.4 Estructura recomendada de GitHub

```
./
|-- README.md
|-- docs/
|   |-- Reto2_Seguridad_Perimetral_IoT_Entrega_Final.docx
|   |-- c4_nivel3_componentes.png
|   `-- flujo_funcional_prototipo.png
-- flows/
|   `-- reto2_nodered_flow.json
-- scripts/
|   `-- sensor_sim.py
```

```
|-- data/
|   |-- demo.sqlite
|   |-- media/
|       |-- demo_reto2_5min.mp4
```

7. Conclusiones

El Reto 2 queda resuelto de forma integral: se aplican tres patrones arquitectonicos con justificacion funcional, se descompone la solucion en componentes internos mediante un C4 nivel 3 coherente, se especifica un prototipo funcional ejecutable con sensores reales o simulados y se documentan trade-offs y guion de demostracion. La propuesta mantiene un balance apropiado entre sencillez de laboratorio y criterio arquitectonico profesional.

La arquitectura resultante es suficientemente robusta para continuar hacia una siguiente fase de documentacion, evaluacion y despliegue, pero al mismo tiempo mantiene el costo y la complejidad controlados para una entrega academica. El sistema se entiende, se implementa y se defiende con claridad porque cada componente, contrato y decision queda explicitado dentro de este mismo documento.

Anexo A. Comandos MQTT de prueba

Suscripcion general para observar el flujo

```
mosquitto_sub -h localhost -t "iot/perimetro/#" -v
```

Publicacion manual de un evento valido

```
mosquitto_pub -h localhost -t "iot/perimetro/zona1/movimiento/raw" \
-m '{"deviceId":"esp32cam-z1", "zone":"zona1", "eventType":"motion", "motion":true, "confidence":0.84, "imageRef":"capture_20260414_203000.jpg", "timestamp":"2026-04-14T20:30:00-05:00"}'
```

Anexo B. Script Python de publicacion simulada

sensor_sim.py

```
import json
from datetime import datetime
import random
import paho.mqtt.client as mqtt

BROKER = "localhost"
TOPIC = "iot/perimetro/zona1/movimiento/raw"

payload = {
    "deviceId": "esp32cam-z1",
    "zone": "zona1",
    "eventType": "motion",
    "motion": True,
    "confidence": round(random.uniform(0.68, 0.92), 2),
    "imageRef": f"capture_{datetime.now():%Y%m%d_%H%M%S}.jpg",
    "timestamp": datetime.now().astimezone().isoformat()
}

client = mqtt.Client(client_id="sim-z1")
client.connect(BROKER, 1883, 60)
client.publish(TOPIC, json.dumps(payload), qos=1)
client.disconnect()
```

Anexo C. Function node sugerido para score y validacion

function score

```
const e = msg.payload;
const now = Date.now();
const last = flow.get(`last_${e.zone}`) || 0;
let score = e.motion ? 0.55 : 0.0;

if (e.imageRef) score += 0.10;
if ((now - last) <= 10000 && last !== 0) score += 0.20;

const hour = new Date().getHours();
if (hour >= 22 || hour <= 5) score += 0.15;
if ((now - last) < 5000 && last !== 0) score -= 0.20;

e.finalScore = Number(score.toFixed(2));
e.validated = e.finalScore >= 0.70;
msg.payload = e;

if (e.validated) {
    flow.set(`last_${e.zone}`, now);
}

return msg;
```

Anexo D. Recomendaciones finales de montaje

- Mantener el mismo contrato JSON tanto para simulador como para ESP32-CAM con el fin de no modificar Node-RED entre demo y hardware real.
- Versionar el flujo Node-RED exportado en GitHub para que la demo pueda reconstruirse facilmente.
- Configurar un dataset de evidencia pequeno y nombres de zona legibles para que el dashboard sea entendible durante la grabacion.
- No publicar directamente al dashboard desde el sensor; todo debe pasar por el Gateway para conservar consistencia arquitectonica.